

Relazione Homework 1

Stack Buffer Overflow

Studenti: *Andrea Erpini 2082488, Eleonora Conti 2082947*

Indice

1. Strumenti e Software utilizzati	2
2. Progettazione vulnerabilità	2
Meccanismo della Vulnerabilità	2
3. Ispezione dei Registri e Calcolo dell'Offset	2
4. Sviluppo dell'Exploit e Struttura del Payload	3
5. Fase d'Attacco	4
6. Problemi Riscontrati e Soluzioni Adottate	4
6.1 Lo Shellcode non apriva la finestra CMD	4
6.2 Indirizzo dell'API WinExec Errato	4
6.3 Crash dell'Applicazione dopo l'Apertura della Shell	5
6.4 Presenza di Caratteri Bad Characters nel Payload	5
6.5 Gestione Little-Endian	5
7. Escalation dei Privilegi (Parte Opzionale)	6
7.1 Obiettivo	6
7.2 Scelta dell'Exploit Locale	6
7.2.1 Fonti e Reperimento di KiTrap0D	6
7.2.2 Note sulla consegna della parte opzionale	7
7.3 Modifica del Payload per Concatenazione	7
7.4 Prerequisiti Operativi	7
7.5 Fase di Attacco Completa	8
7.6 Verifica dell'elevazione dei Privilegi	8
7.7 Problemi Riscontrati e Soluzioni Adottate	9
7.7.1 Mancanza della DLL di supporto	9
7.7.2 Variabile d'ambiente non aggiornata	9

1. Strumenti e Software utilizzati

Per questo progetto è stato utilizzato il sistema operativo Windows XP Professional 32-bit Service Pack 3 non patchato con locazione in Italia.

Al fine di riprodurre lo scenario didattico richiesto dalla traccia, sono state rimosse delle mitigazioni di sicurezza sia a livello OS che in fase di compilazione:

- **A livello del Sistema Operativo:** è stata disattivata globalmente la protezione **DEP (Data Execution Prevention)** per consentire l'esecuzione dello shellcode posizionato nell'area di memoria dello stack.
- **A livello Compilatore:** in fase di generazione dell'eseguibile tramite GCC, è stata disabilitata l'introduzione dei controlli di integrità dello stack frame **Stack Canaries** tramite questo comando: `"gcc -fno-stack-protector -o guess_game.exe guess_game.c"`

Gli strumenti software utilizzati per l'attività sono:

- **Immunity Debugger:** Per l'analisi dinamica del crash, l'ispezione dei registri della CPU, il calcolo dell'offset e l'individuazione degli indirizzi di memoria statici all'interno delle DLL di sistema.
- **Visual Studio Code / Python 3:** Per l'analisi del codice sorgente, lo sviluppo e la generazione dei file bin finali (payload.bin e payload2.bin).

2. Progettazione vulnerabilità

È stato creato un piccolo gioco in c dove bisogna inserire un nome utente e poi cercare di indovinare il numero che ha pensato l'oracolo, ma questo gioco presenta una grave vulnerabilità di Stack Buffer Overflow situata all'interno della funzione di input `get_name()` che è visibile durante la prima interazione con l'utente.

Meccanismo della Vulnerabilità

Alla funzione precedentemente citata viene allocata sullo stack una variabile locale denominata buffer di dimensione fissa pari a 64 byte. La funzione di lettura successiva, `fgets(buffer, 1024, stdin)`, è invece configurata per accettare un flusso di input lungo fino a 1024 byte. Poiché la dimensione massima consentita in lettura supera la capacità strutturale della variabile ospitante, e in totale assenza di controlli preventivi sui limiti (bounds checking), un input eccedente andrà a sovrascrivere lo stack frame corrente, corrompendo i dati adiacenti.

3. Ispezione dei Registri e Calcolo dell'Offset

Per identificare i vari registri dello stack al momento dell'errore e mappare il comportamento che assumono questi ultimi, l'eseguibile è stato monitorato all'interno di Immunity Debugger applicando una metodologia di analisi incrementale:

- **Primo tentativo:** inizialmente è stata inviata al programma una stringa molto lunga composta esclusivamente dal carattere A (\x41 in hex). Questo input ha provocato l'arresto anomalo dell'applicazione. Ispezionando lo stato dei registri nel debugger, è stato rilevato che il registro EIP puntava al valore 41414141. Questo ha confermato la vulnerabilità: la CPU ha tentato di interpretare la sequenza di A come un indirizzo

di memoria a cui saltare, fallendo e causando il crash. L'uso di un unico carattere ripetuto, tuttavia, non permetteva ancora di quantificare la distanza esatta del punto di sovrascrittura.

- **Secondo tentativo:** Per determinare quanti byte separassero l'inizio del buffer dal Return Address è stata iniettata una seconda stringa combinata da un blocco iniziale di sole A seguito da una sequenza finale del carattere B (\x42 in hex). Al momento del crash, il registro EIP ha mostrato il valore esadecimale 42424242. Questa dimostrazione visiva ha permesso di mappare lo stack con precisione.
- **Calcolo Finale dell'Offset:** L'ispezione combinata del dump di memoria e dei registri ha stabilito che sono necessari esattamente 76 byte per saturare il buffer da 64 byte e i successivi dati di allineamento e controllo dello stack frame compreso il Saved EBP. I 4 byte immediatamente successivi, ovvero dal byte 77 al byte 80, occupano lo spazio riservato all'EIP. Qualsiasi valore inserito in questa precisa finestra sovrascriverà l'indirizzo di ritorno che la CPU caricherà nel proprio flusso di esecuzione non appena la funzione `get_name()` incontrerà l'istruzione di ritorno RET.

4. Sviluppo dell'Exploit e Struttura del Payload

L'exploit si basa sullo sfruttamento di puntatori statici all'interno delle librerie dinamiche di sistema di Windows XP. Tramite Immunity Debugger è stata analizzata la libreria **kernel32.dll**. La scelta è ricaduta su questo modulo poiché, trattandosi di una libreria nativa essenziale del sistema operativo, garantisce stabilità e posizionamento fisso in memoria a causa dell'assenza della mitigazione ASLR, non prevista in questa release di Windows. Il payload finale iniettato è stato strutturato secondo la seguente sequenza di byte:

- **Padding (76 byte):** Composto da una sequenza di caratteri A (\x41) per riempire lo spazio nello stack e raggiungere esattamente la soglia dell'EIP.
- **Sovrascrittura EIP (4 byte):** È stato inserito l'indirizzo statico 0x7C86467B situato all'interno di kernel32.dll, il quale punta all'istruzione JMP ESP. Al momento dell'esecuzione di RET, la CPU caricherà questo indirizzo ed eseguirà il salto, indirizzando istantaneamente il flusso di esecuzione verso il registro ESP che in quel momento punta all'inizio dello spazio di memoria successivo.
- **NOP Sled (4 byte):** Una sequenza di \x90 inserita come cuscinetto di sicurezza per garantire la stabilità del salto prima dell'esecuzione dello shellcode vero e proprio.
- **Shellcode Custom:** Sequenza in codice macchina a 32-bit sviluppata per invocare le API di Windows WinExec e ExitProcess. Nello specifico, lo shellcode forza l'apertura di una nuova istanza indipendente del terminale tramite il comando
cmd /c start cmd
permettendo così di interagire con la shell target senza bloccare il processo padre.

5. Fase d'Attacco

1. Lo script in Python "payloadBase.py" viene eseguito per assemblare i componenti sopra descritti e generare il file binario grezzo payload.bin.
2. Viene aperto il prompt dei comandi di Windows nella directory dell'applicazione.
3. Viene avviato l'eseguibile reindirizzando il file binario sullo standard input:

guess_game.exe < payload.bin

4. La funzione fgets acquisisce il payload, l'EIP punta a JMP ESP, la CPU salta sullo stack ed esegue lo shellcode.
5. **Risultato:** il processo vulnerabile termina in modo pulito grazie a ExitProcess e sullo schermo si apre una nuova finestra di cmd.exe interattiva con i privilegi del programma originale.

6. Problemi Riscontrati e Soluzioni Adottate

Durante lo sviluppo dell'exploit sono stati affrontati diversi ostacoli tecnici, risolti attraverso un processo di analisi e correzione iterativo.

6.1 Lo Shellcode non apriva la finestra CMD

- **Problema:** Lo shellcode iniziale invocava direttamente la funzione WinExec("cmd", 1). Il risultato era il congelamento dell'applicazione e la mancata comparsa di una shell interattiva utilizzabile.
- **Analisi:** Poiché guess_game.exe viene eseguito reindirizzando l'input da file < payload.bin, il canale di standard input stdin del processo rimane permanentemente collegato al file binario stesso. Di conseguenza, il nuovo processo cmd.exe, ereditando la console corrente dal processo padre, tentava di leggere i comandi da payload.bin invece che dalla tastiera dell'utente, fallendo.
- **Soluzione:** È stata modificata la stringa di comando inserita nello shellcode, passandola da "cmd" a "cmd /c start cmd". Il comando interno start di Windows impone al sistema operativo di istanziare un nuovo processo e una nuova finestra del terminale totalmente indipendente, svincolata dal reindirizzamento dello stdin originario.

6.2 Indirizzo dell'API WinExec Errato

- **Problema:** Inizialmente nello script Python era stato inserito l'indirizzo 0x7C8633AD per la funzione WinExec. Durante l'esecuzione, l'exploit falliva sistematicamente provocando un crash generico all'interno di kernel32.dll all'offset 000633af.
- **Analisi:** L'indirizzo inserito non corrispondeva all'effettivo punto di ingresso della funzione API, bensì a una locazione intermedia casuale all'interno del codice della DLL. Quando lo shellcode eseguiva l'istruzione CALL EBX, la CPU tentava di decodificare istruzioni non allineate o dati grezzi, corrompendo l'esecuzione.
- **Soluzione:** È stata effettuata una verifica mirata in Immunity Debugger impostando un breakpoint esplicito sul simbolo della funzione tramite il comando:

“bp WinExec”. La corretta risoluzione dei simboli ha restituito l'indirizzo reale e accurato, pari a 0x7C8623AD, che è stato prontamente aggiornato nello script.

6.3 Crash dell'Applicazione dopo l'Apertura della Shell

- **Problema:** l'exploit generava correttamente la shell, ma subito dopo l'applicazione principale guess_game.exe andava in crash visualizzando la finestra di errore standard di Windows.
- **Analisi:** Una volta completata l'esecuzione della funzione WinExec, in assenza di istruzioni di terminazione esplicite, la CPU continuava a scorrere la memoria dello stack in sequenza lineare, tentando di eseguire i byte successivi che non erano altro che residui del payload o dati casuali come se fossero istruzioni operative, portando inevitabilmente a una Access Violation.
- **Soluzione:** Lo shellcode è stato ottimizzato introducendo in coda una seconda chiamata API diretta alla funzione ExitProcess(0). In questo modo, una volta lanciato il terminale separato, il processo padre guess_game.exe viene terminato dal sistema operativo in modo controllato e pulito, senza sollevare eccezioni o anomalie visibili a schermo.

6.4 Presenza di Caratteri Bad Characters nel Payload

- **Problema:** Nelle prime iterazioni dello shellcode, il payload veniva troncato e non riusciva a sovrascrivere l'EIP, impedendo il dirottamento del flusso.
- **Analisi:** Il meccanismo di acquisizione tramite fgets interrompe la lettura solo in presenza del carattere di Newline (\x0A) o di EOF, mentre è in grado di leggere e copiare in memoria il byte nullo (\x00) senza interrompersi. Tuttavia, \x00 deve essere tassativamente evitato all'interno del payload poiché le funzioni API di Windows (come WinExec), invocate successivamente dallo shellcode, trattano gli argomenti come stringhe standard del C (null-terminated). La presenza di un byte nullo non pianificato causerebbe quindi il troncamento dei parametri passati all'API, impedendo l'esecuzione del comando.
- **Soluzione:** Lo shellcode è stato interamente programmato per evitare l'utilizzo di byte nocivi. Le operazioni che normalmente avrebbero generato degli zeri come l'azzeramento di un registro sono state sostituite da istruzioni logiche equivalenti. È stato inoltre verificato preventivamente che nessuno degli indirizzi di memoria statici impiegati ovvero: JMP ESP, WinExec e ExitProcess contenesse al proprio interno i byte vietati \x00, \x0A o \x0D.

6.5 Gestione Little-Endian

- **Problema:** Inserendo l'indirizzo di ritorno dell'EIP scrivendo i byte nell'ordine logico di lettura 7C 86 46 7B, l'applicazione andava in crash senza saltare all'istruzione JMP ESP.
- **Analisi:** L'architettura hardware x86 della CPU memorizza e legge i dati in modalità Little-Endian. Questo significa che all'interno della memoria il byte meno significativo deve essere posizionato per primo. Inserire l'indirizzo nell'ordine standard induce la CPU a interpretare un indirizzo di memoria invertito e completamente errato.

- **Soluzione:** All'interno dello script Python, il problema è stato risolto automatizzando la conversione strutturale tramite il modulo nativo struct. L'utilizzo del costrutto: `Python struct.pack("<I", JMP_ESP)` dove il carattere < specifica esplicitamente la codifica Little-Endian garantisce che il valore 0x7C86467B venga scritto nel file binario nell'esatta sequenza di byte richiesta dall'hardware: \x7B\x46\x86\x7C
-

7. Escalation dei Privilegi (Parte Opzionale)

7.1 Obiettivo

Dato che Windows XP Professional SP3 è privo di patch correttive e sono state disabilitate le mitigazioni (DEP e Stack Canaries), si è rivelato possibile concatenare la vulnerabilità di buffer overflow con un exploit locale a livello kernel per ottenere un accesso completo al sistema.

7.2 Scelta dell'Exploit Locale

Per condurre l'attività di escalation locale è stato selezionato l'exploit noto come KiTrap0D. Questa tecnica sfrutta una falla nel gestore delle eccezioni #GP (nt!KiTrap0D) del kernel Windows XP. La vulnerabilità consente a un processo operante in modalità utente di manipolare il Thread Information Block del sottosistema Virtual DOS Machine, inducendo il kernel a ripristinare un execution stack interamente controllato dall'attaccante. Ciò determina l'esecuzione di codice arbitrario in modalità kernel (Ring 0) con la conseguente apertura di un prompt dei comandi con privilegi NT AUTHORITY\SYSTEM.

Il sistema bersaglio è risultato esposto all'attacco in quanto privo della patch specifica, rilasciata da Microsoft per correggere tale anomalia. La verifica preliminare è stata eseguita tramite il comando "wmic qfe get HotFixID", il cui output ha evidenziato esclusivamente la presenza dell'aggiornamento storico Q147222, confermando l'assenza di contromisure di sicurezza moderne.

7.2.1 Fonti e Reperimento di KiTrap0D

- **Repository dettagli exploit:** [Microsoft Windows NT/2000/2003/2008/XP/Vista/7 - 'KiTrap0D' User Mode to Ring Escalation \(MS10-015\)](#)

Per quanto concerne le componenti operative necessarie alla conduzione dell'attacco, il pacchetto contenente l'eseguibile **vdmallowed.exe** e il modulo **vdmexploit.dll** è stato acquisito tramite il repository dei binari di Exploit-DB al seguente indirizzo di download:

- **Download pacchetto binario:** <https://gitlab.com/exploit-database/exploitdb-bin-splotts/-/raw/main/bin-splotts/11199.zip>

7.2.2 Note sulla consegna della parte opzionale

Si precisa che l'exploit **KiTrap0D (vdmallowed.exe)** e il relativo modulo ausiliario **vdmexploit.dll** non sono stati inclusi nel materiale allegato per la consegna, in quanto si tratta di vettori di attacco a livello kernel ampiamente noti e classificati come minacce software dai sistemi di protezione. La loro distribuzione comporterebbe una violazione delle politiche di sicurezza e dei termini di servizio, impedendo il corretto trasferimento dei file. Tuttavia, il pacchetto completo è regolarmente disponibile e funzionante nell'ambiente di laboratorio (macchina virtuale con Windows XP) impiegato per lo svolgimento e la documentazione dell'attività.

7.3 Modifica del Payload per Concatenazione

Al fine di integrare l'exploit locale all'interno della catena d'attacco, lo shellcode originario è stato opportunamente modificato affinché, in luogo dell'invocazione diretta di `cmd.exe`, provveda a eseguire l'eseguibile precompilato `vdmallowed.exe` (corrispondente all'exploit **KiTrap0D**) mediante una chiamata mirata alla funzione API `WinExec` contenuta nella libreria di sistema `kernel32.dll`.

La stringa testuale "`vdmallowed.exe`" è stata strutturata dinamicamente all'interno dello stack attraverso una sequenza definita di istruzioni `push` a blocchi di 4 byte per rispettare la rappresentazione in memoria imposta dall'architettura Little-Endian che è stata creata tramite il file `escalation.py`. Poiché la stringa è lunga 14 caratteri, non allineata a 4 byte, l'ultimo blocco pushato è stato riempito con byte di padding `\x90` (NOP) al posto dei byte nulli, per evitare la presenza di bad characters nello shellcode. Il null terminator è stato poi inserito dinamicamente a runtime tramite l'istruzione

```
mov byte [eax+14], cl
```

sfruttando il registro `ECX` precedentemente azzerato con `xor ecx, ecx`.

L'evoluzione sequenziale della catena d'attacco si sviluppa secondo le seguenti fasi operative:

1. L'applicazione `guess_game.exe` elabora il flusso di input corrompendo lo stack frame e sovrascrivendo l'EIP con l'indirizzo di puntamento a `JMP ESP` (pari a `0x7C86467B`).
2. La CPU esegue il salto logico verso l'area dello stack avviando lo shellcode, il quale richiama l'API `WinExec` impostando l'esecuzione dell'eseguibile `vdmallowed.exe`.
3. Il binario `vdmallowed.exe` viene inizializzato ereditando inizialmente il contesto di sicurezza dell'utente locale `testuser`.
4. La routine di exploit del kernel integrata in `vdmallowed.exe` esegue l'elevazione dei privilegi a livello `SYSTEM`, istanziando una nuova console interattiva di `cmd.exe`

7.4 Prerequisiti Operativi

Affinché l'API `WinExec` individui correttamente i binari, è tassativo che l'eseguibile `vdmallowed.exe` e il relativo modulo ausiliario `vdmexploit.dll` siano collocati nella stessa directory di lavoro dell'applicazione vulnerabile `guess_game.exe`. L'assenza di tale libreria

causa l'interruzione immediata del thread di attacco, visualizzando a schermo un errore di caricamento del modulo iniettato.

7.5 Fase di Attacco Completa

La conduzione dell'attacco combinato si articola attraverso la seguente sequenza metodologica:

- Generazione dell'exploit in formato binario grezzo mediante l'esecuzione dello script Python aggiornato.
- Disposizione delle componenti vdmallowed.exe e vdmexploit.dll all'interno della cartella contenente l'eseguibile target.
- Iniezione del payload avviando l'applicazione tramite il reindirizzamento dello standard input con il comando `guess_game.exe < payload2.bin`.
- Chiusura controllata e regolare del processo originario `guess_game.exe` mediante la chiamata all'API `ExitProcess`.
- Inizializzazione automatica del binario `vdmallowed.exe` con conseguente esecuzione dei vettori di attacco a livello kernel.
- Apertura di una nuova sessione interattiva del terminale `cmd.exe` operante con i massimi privilegi di `NT AUTHORITY\SYSTEM`.

7.6 Verifica dell'elevazione dei Privilegi

La convalida dell'effettivo livello di autorizzazione conseguito su ambiente Windows XP SP3 è stata svolta mediante una prima interrogazione della variabile d'ambiente `echo %username%` che ha restituito l'utenza `testuser`. Tale stringa, tuttavia, viene definita all'inizializzazione della shell e non riflette gli aggiornamenti dinamici applicati al token di sicurezza primario, rivelandosi un indicatore non attendibile per certificare i privilegi reali.

L'evidenza definitiva dell'avvenuta escalation è stata ottenuta inoltrando una direttiva esplicita al Task Scheduler di sistema tramite il comando:
Istruzione inviata al terminale:

at 17:00 /interactive "cmd.exe"

Mentre l'esecuzione di tale comando in una sessione non privilegiata solleva un'eccezione di accesso negato, nel prompt derivante dall'exploit la richiesta viene elaborata con successo. Questo comportamento attesta in modo univoco che il token attivo detiene i massimi diritti amministrativi tipici dell'account `SYSTEM` necessari per interagire con il gestore delle pianificazioni.

A ulteriore riprova della riuscita dell'attacco, si è proceduto a effettuare una verifica di scrittura forzata all'interno di una cartella protetta dal sistema operativo:
Comando di scrittura eseguito:

echo test > C:\WINDOWS\System32\prova.txt

Il perfetto completamento della scrittura del file ha fornito la dimostrazione materiale del conseguimento di privilegi amministrativi superiori.

7.7 Problemi Riscontrati e Soluzioni Adottate

7.7.1 Mancanza della DLL di supporto

- **Problema:** Durante la prima fase di test, l'esecuzione del file vdmallowed.exe si interrompeva in modo anomalo segnalando l'impossibilità da parte del thread di attacco di caricare la DLL iniettata, inibendo il processo di escalation.
- **Analisi:** Le verifiche sul comportamento dell'exploit hanno evidenziato che la versione precompilata di KiTrap0D necessita della presenza costante della libreria vdmexploit.dll per finalizzare l'iniezione di codice all'interno del processo DEBUG.EXE nel sottosistema NTVDM. Tale risorsa non risultava allocata all'interno della cartella corrente.
- **Soluzione:** è stata inserita la libreria richiesta nella cartella dove risiede l'exploit

7.7.2 Variabile d'ambiente non aggiornata

- **Problema:** Ad attacco concluso, l'interrogazione testuale della variabile echo %username% persisteva nel mostrare la stringa testuser, sollevando perplessità sulla reale efficacia delle operazioni di escalation.
- **Analisi:** La variabile d'ambiente %username% viene configurata in modo statico unicamente all'atto della creazione del processo cmd.exe. L'exploit KiTrap0D genera la nuova shell ereditando l'ambiente del profilo utente originale ma associandovi un differente token di sicurezza elevato; di conseguenza, la variabile testuale non risulta allineata con l'effettivo livello di autorizzazione attivo.
- **Soluzione:** L'effettiva elevazione è stata verificata sperimentalmente eseguendo direttive privilegiate tramite il servizio at e conducendo test di scrittura all'interno della directory protetta C:\WINDOWS\System32, i quali hanno confermato l'acquisizione completa dei diritti di amministratore di sistema indipendentemente dal valore non aggiornato della variabile ambientale.